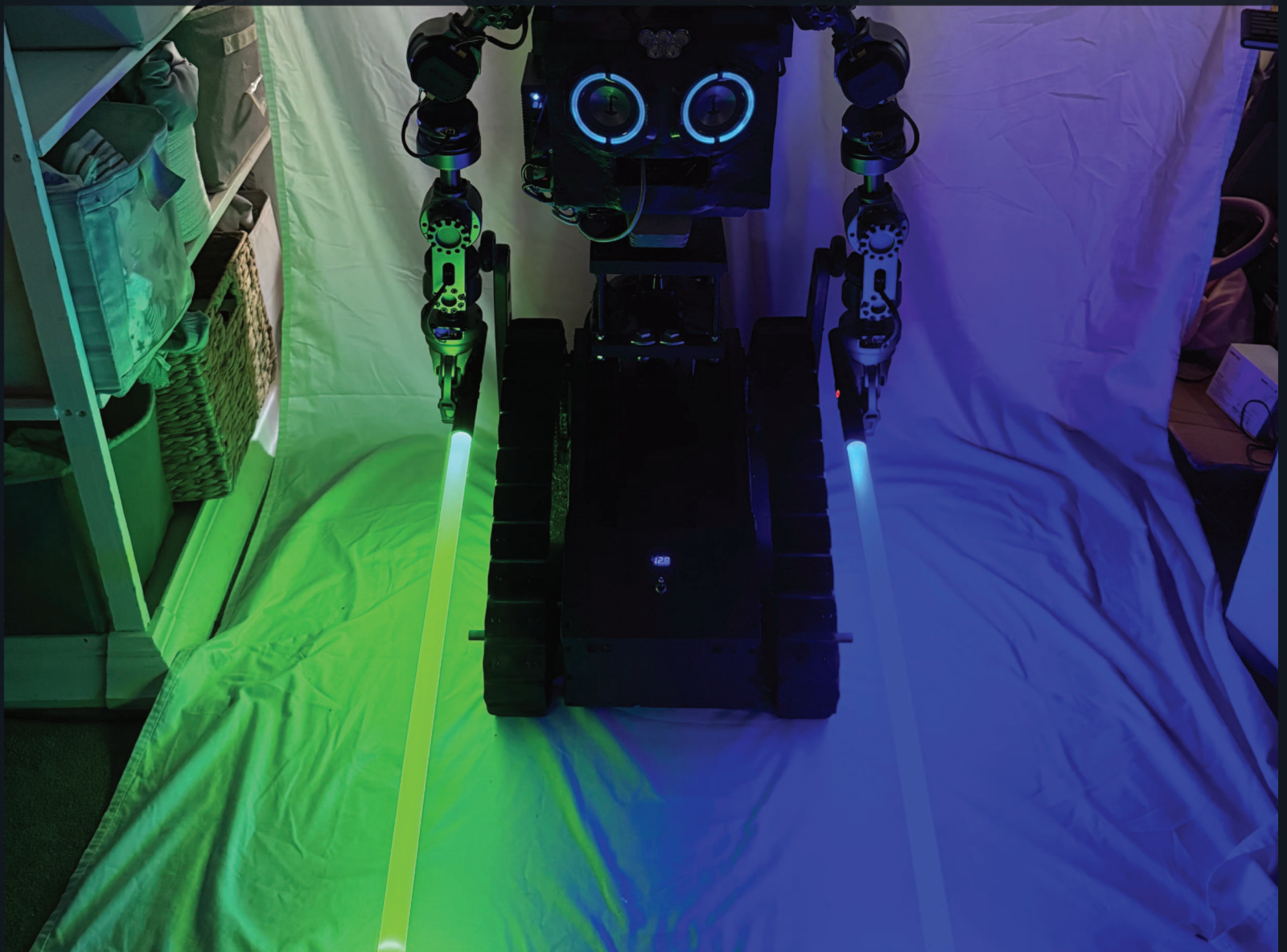

BUILDING R.O.B. - VOLUME 2

CIRCUITS AND SIGNALS

Arduino lessons from ROB's tracked base



A LOW-VOLTAGE LEARNING GUIDE FOR MAKERS AGES 10-14

Rodolfo Aramayo / OrbitusRobotics LLC - First Maker Faire Edition

Electricity is powerful. Information makes it useful.

In this volume, ROB's real base firmware becomes a map for safe classroom experiments.

You will learn how circuits carry energy, how pins carry commands, how a serial message represents motion, and why a robot must stop when messages vanish. The examples are based on `ROBOT_CEREBELLULAR_BASE_APP.ino`, but the classroom builds use LEDs, a simulator, or a small current-limited robotics kit.

Do not connect ROB's full-size batteries, motors, charger, or actuator from these pages. Photographs show where ideas came from; they are not wiring diagrams.

Copyright © 2026 OrbitusRobotics LLC. All rights reserved. Rodolfo Aramayo is the author and photographer. Photographs are from the private ROB build archive unless otherwise noted. Generated covers, frontispieces, story scenes, and conceptual teaching plates are original project illustrations derived from or inspired by ROB reference photographs. They are illustrations, not documentary photographs, technical drawings, or evidence of the as-built configuration. Source-code licenses remain separate and repository-specific. This independent educational project is not affiliated with or endorsed by any film studio or franchise owner. Product and company names remain the property of their respective owners.

First evidence-based draft: 2 August 2026. This historical engineering edition distinguishes documented facts, builder recollection, experiments, plans, and facts not established by the available evidence.

The Building R.O.B. library

VOLUME 1	<i>Meet ROB: A Robot Is a Team of Systems</i> - ages 8-12
VOLUME 2	<i>Circuits and Signals with ROB</i> - ages 10-14
VOLUME 3	<i>Motion Workshop: Treads, Gears, and Fabrication</i> - ages 10-15
VOLUME 4	<i>Mission Control: Arduino, Mac, Networks, and Vision</i> - ages 12-16
VOLUME 5	<i>AI, Robotics, and Codex: Directing, Verifying, and Owning AI-Built Systems</i> - advanced makers
VOLUME 6	<i>Dual-Arm Robotics: AMBER B1, URDF, CAN, and Ubuntu</i> - advanced makers
VOLUME 7	<i>Engineering ROBControllerVision: Swift, Spatial Input, Video, and Speech</i> - Swift developers
VOLUME 8	<i>Engineering Cerebro: Perception, Models, H.264, and Robot Coordination</i> - software engineers
MANUAL	<i>Building R.O.B.: The Complete Maker's Field Manual</i> - advanced builders

HOW TO USE THIS BOOK

Bring a notebook. For each activity, write your prediction, observation, and explanation. If the result surprises you, the experiment worked: you found something worth learning.

ONE CURRENT ARDUINO, THREE HISTORICAL SKETCHES

The ROBArduino archive preserves Base, Torso, and Head sketches from an earlier three-Arduino design. The builder reports that present-day ROB uses **only the Base sketch**. This book therefore treats Base behavior as the current low-level firmware reference and labels Torso and Head behavior as retired historical evidence. Source files cannot prove which binary is flashed today; record the board identity, firmware hash, build environment, and upload date before operation.

TWO AMBER B1 ARMS, ONE DOCUMENTED CONTROL CHAIN

ROB carries two seven-joint AMBER B1 arms. The repository snapshot separates their Ubuntu-side cores as L-10 and R-11: network clients use UDP or LCM, each core binds a stable CAN interface, and CAN carries commands and feedback between that core and its arm actuators. The `AmberHomeFolder` tree is evidence from one Ubuntu machine, not a deployable image; never copy its credentials, logs, virtual environment, or machine-specific identifiers. Volume 6 documents the inspected protocol, URDF models, reproducible setup, verification gates, and unresolved configuration differences.

HOW TO FIND THE FILES

Open the public repositories on GitHub and follow each printed repository-relative path: <https://github.com/RudyAramayo/ROBBooks>, <https://github.com/RudyAramayo/ROBArduino>, <https://github.com/RudyAramayo/Cerebro>, <https://github.com/RudyAramayo/ROBController>, <https://github.com/RudyAramayo/ROBControllerVision>, <https://github.com/RudyAramayo/Amber-HomeFolder>, and <https://github.com/RudyAramayo/ROBTrainingGames>. The website is published separately at <https://github.com/OrbitusRoboticsWebSite/0Robotics>. See <https://github.com/RudyAramayo/ROBBooks/blob/main/ROB-Books/OPEN-SOURCE-CODE-MAP.md> for clickable repository and file links, license cautions, and renamed-file search instructions. A named archival item without a verified

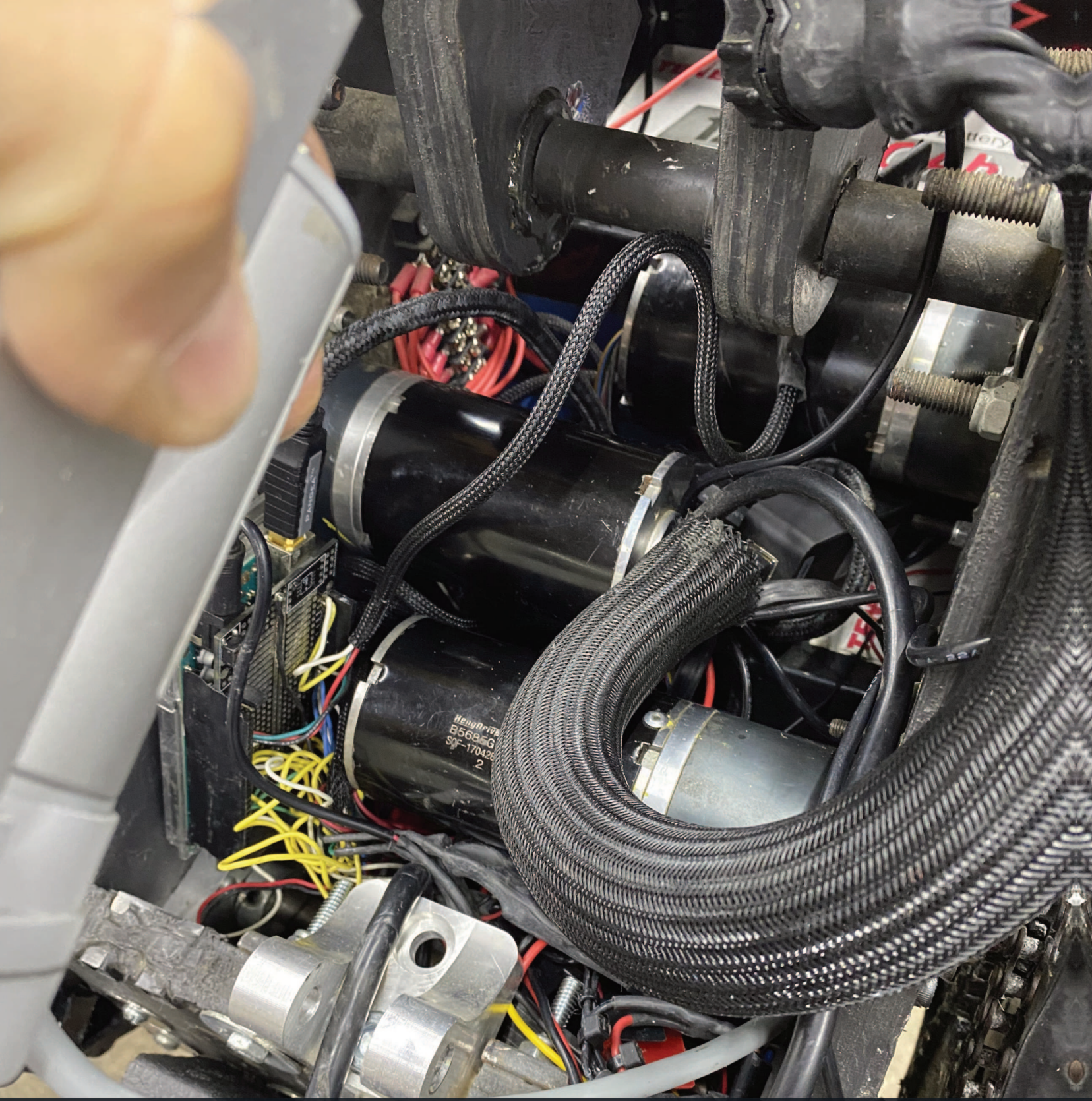
public repository is labeled as evidence, not given an invented URL.

ELECTRICAL LAB BOUNDARY

For learner activities, use only an approved low-voltage classroom supply or the USB-powered Arduino setup selected by your teacher. Add current-limiting resistors to LEDs. Disconnect power before moving wires. ROB's high-current battery and motor circuits are adult-only systems that require fuses, a disconnect, insulated tools, proper wire sizing, and a tested physical emergency stop.

Contents

1	Trace the energy path	2
2	Digital, analog, and PWM	6
3	The base controller's connections	8
4	Decode the seven-field message	13
5	IR distance and the IMU	17
6	Watchdogs and fail-safe states	19
7	Read old code like an engineer	21
8	Build a mental oscilloscope	24
9	Give a toy a new 5 V power path	25
10	Messages need grammar and time	29



MISSION 1

Two networks: power and data

MISSION 1

Trace the energy path

A circuit needs a complete path. A source raises electrical potential, a load changes electrical energy into light, sound, heat, or motion, and a return path completes the loop.

Three ideas help us describe the circuit:

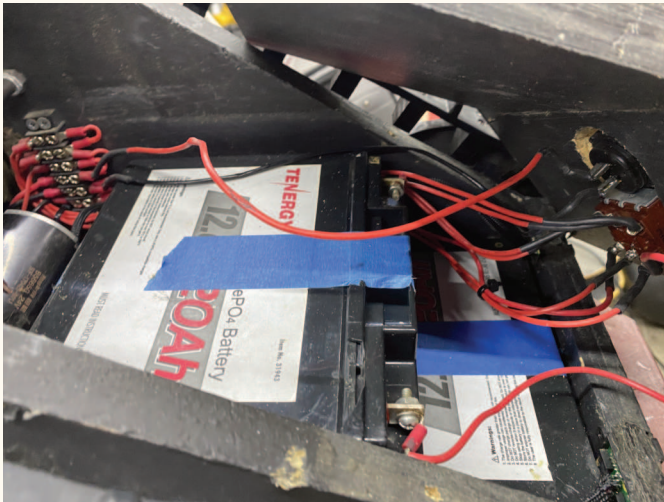
- **Voltage** is electrical potential difference, measured in volts.
- **Current** is the rate of charge flow, measured in amperes.
- **Resistance** describes how strongly a path opposes current, measured in ohms.

Ohm's law, $V = IR$, connects them for a resistor. If an LED branch has 5 V, an LED drop of about 2 V, and a 220 Ω resistor, the estimated current is

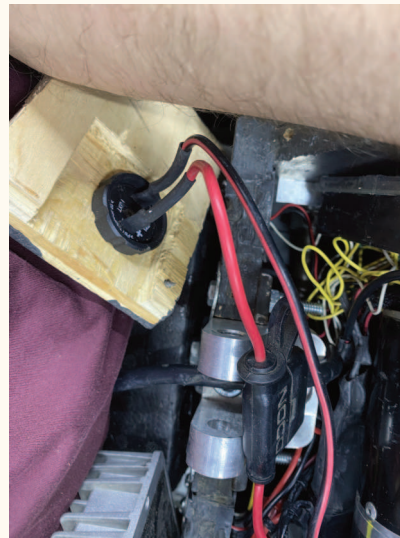
$$I = \frac{5\text{V} - 2\text{V}}{220\Omega} \approx 0.014\text{A} = 14\text{mA}.$$

COMPUTER SCIENCE CONNECTION

A formula is a model. Component tolerances, temperature, supply variation, and the LED's real forward voltage change the result. Measure with a suitable meter and compare the model to the observation.

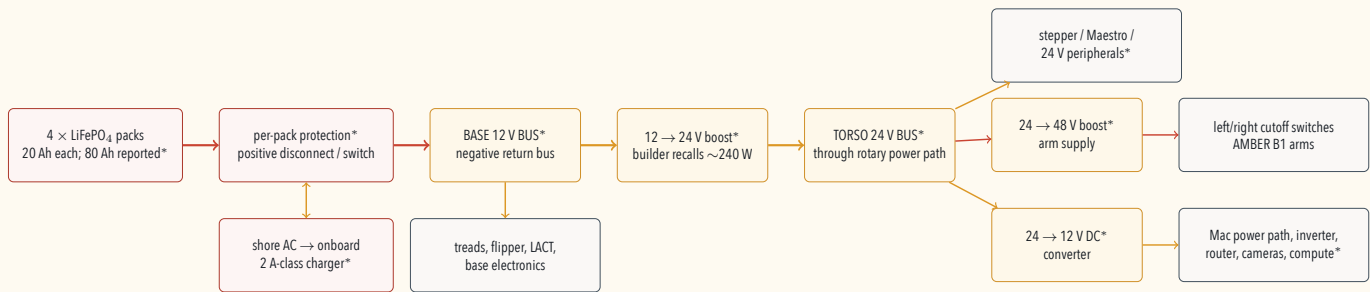


Large batteries and red power leads occupy precious space in the base. The visible arrangement does not prove the complete topology.



A cable crossing a panel needs smooth edges, insulation, strain relief, and room for service.

The builder reports four 20 Ah LiFePO₄ packs, a common negative return bus, and positive leads routed through the front on/off switch to the 12 V bus. A boost stage supplies a 24 V domain, another stage supplies the 48 V arm domain, and a 24-to-12 V converter supplies upper-body 12 V loads. The onboard NOCO charger is recalled as a 2 A-class fallback; that current is too small to assume it can both run every sensor and replenish the full bank.



*Builder-reported architecture. Verify topology, polarity, BMS compatibility, protection, conductor ampacity, converter limits, charger interlocks, and measured loads on the physical robot before energizing.

Ground is a reference, not magic dirt

In a low-voltage robot, **ground** usually names the common reference node used by circuits and signals. Two devices exchanging a voltage signal often need compatible reference levels. Power grounds, noisy motors, sensitive sensors, shields, and chassis connections require a deliberate grounding plan in a large robot.

WHY THIS BOOK HAS NO AS-BUILT POWER TREE

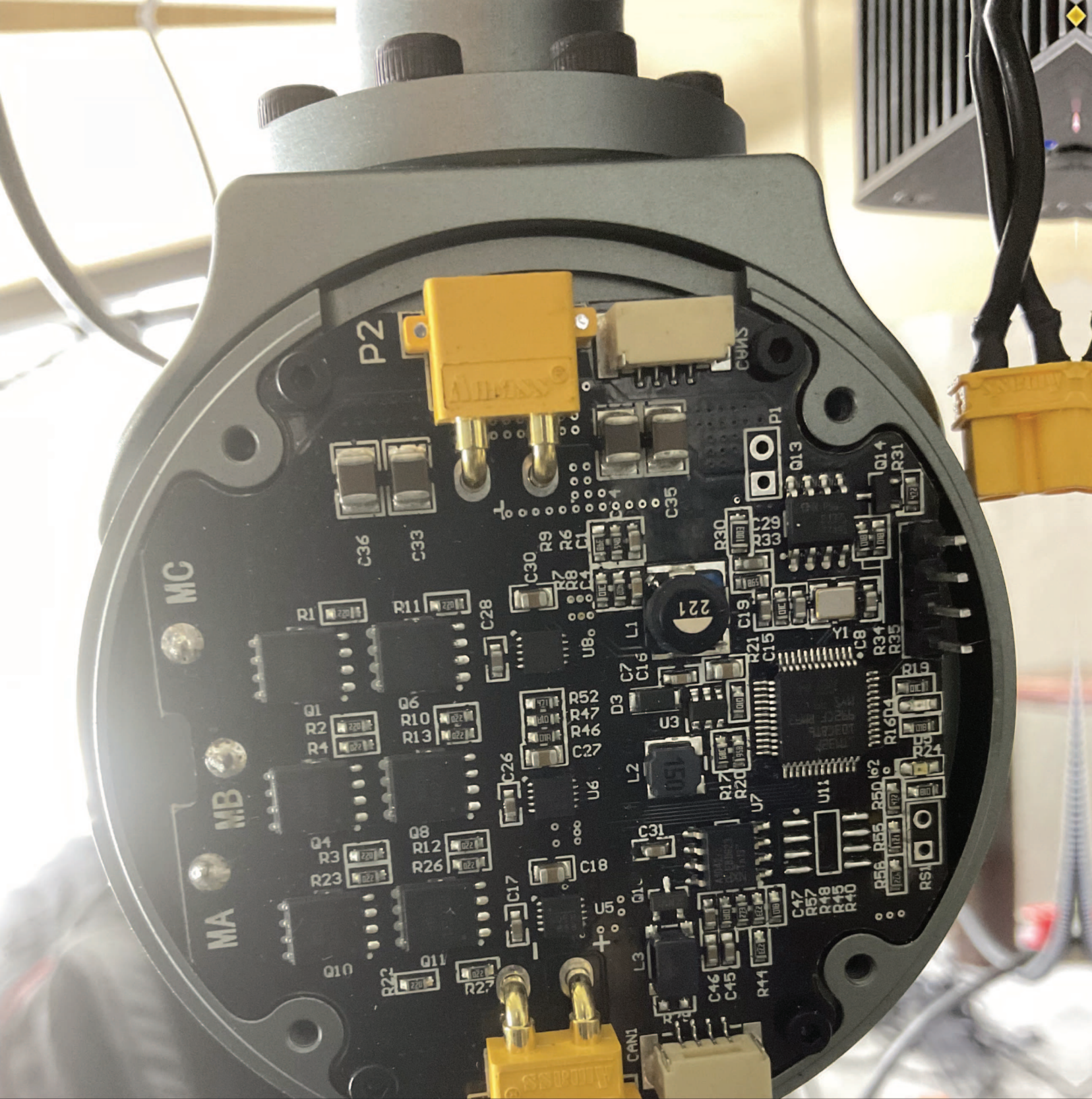
The archive does not contain a verified current schematic, BMS/protection study, conductor schedule, disconnect/E-stop validation, or grounding record. A photograph cannot prove hidden wiring or fault behavior. The diagrams in this volume teach energy and information paths; they are not ROB's construction wiring. Any real robot power system requires a new schematic and qualified adult electrical review.

DRAW BEFORE YOU WIRE

Draw a safe LED circuit with a source, switch, resistor, LED, and return path. Use arrows to show conventional current. Circle the point where the Arduino output signal enters. Ask a mentor to check the drawing before building it.



A visible voltage gauge can help an operator notice change, but it does not replace a schematic, calibrated measurements, battery-management data, or protective devices.



MISSION 2

Pins turn numbers into signals

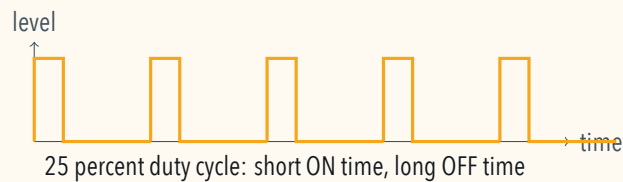
MISSION 2

Digital, analog, and PWM

An Arduino pin can represent information in several ways.

- A **digital output** chooses between two logic states, often called LOW and HIGH.
- An **analog input** measures a voltage and converts it into a number.
- **PWM**, or pulse-width modulation, switches quickly between states. The duty cycle controls the average effect seen by a lamp or motor controller.

PWM is not automatically a lower analog voltage. It is a timed pattern.



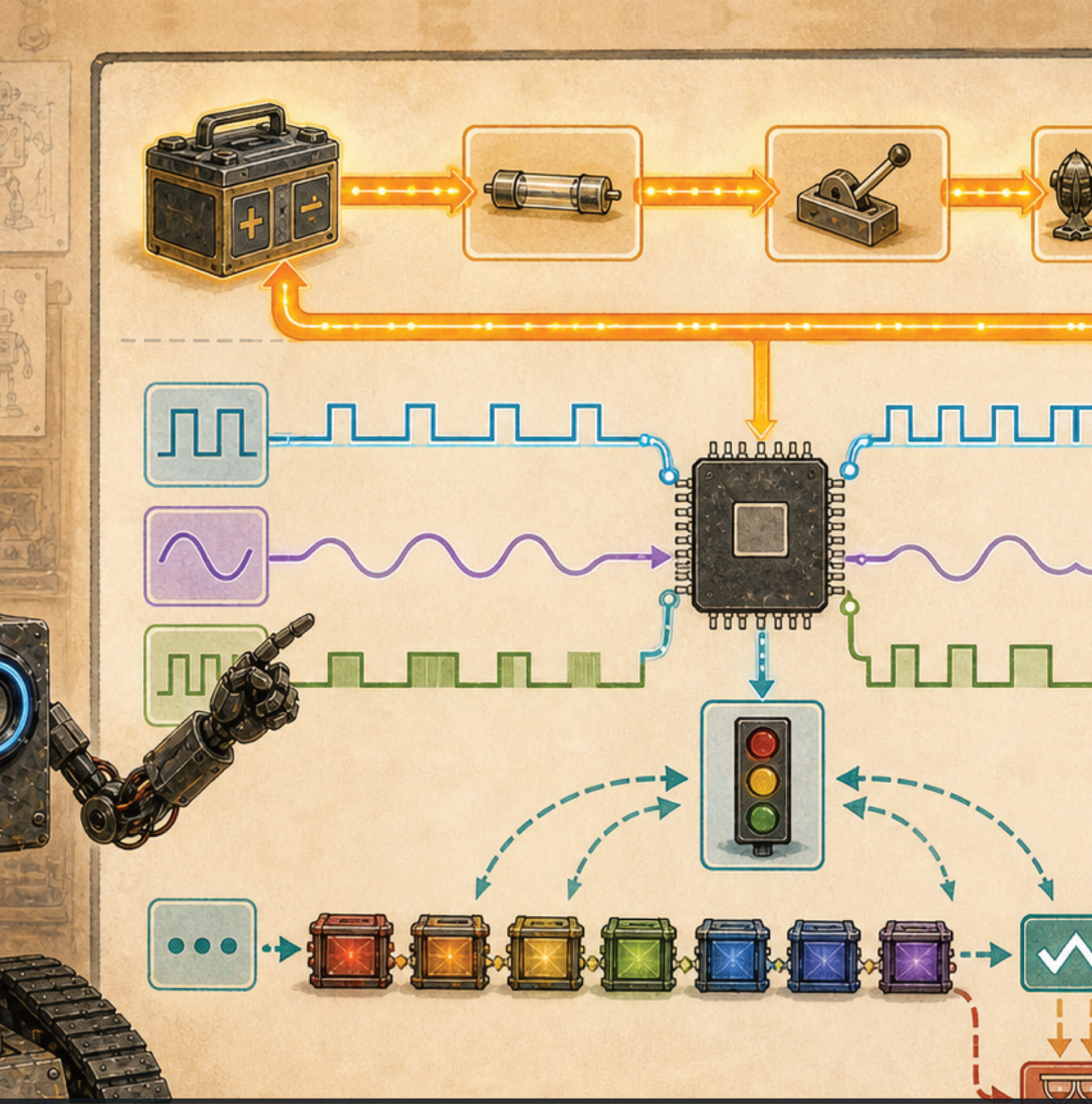
ROB's tread code uses `analogWrite(pin, 255 - abs(command))`. That inversion matters: in this firmware, a larger command magnitude produces a smaller PWM output value. The archived vendor material is restricted, so this public edition does not use it to justify the electrical behavior. Verify the real controller with an authorized current public datasheet and a supervised bench test. Do not assume every motor driver works this way.

LED PWM MISSION

With a mentor-approved Arduino and LED circuit, write three fixed brightness levels using `analogWrite`: 32, 128, and 224. Predict which looks brightest. Then make a slow fade. An LED makes PWM visible without moving a heavy mechanism.

Listing 2.1: A safe LED fade pattern for a classroom PWM pin.

```
1 const int LED_PIN = 9;
2
3 void setup() {
4   pinMode(LED_PIN, OUTPUT);
5 }
6
7 void loop() {
8   for (int level = 0; level <= 255; level += 5) {
9     analogWrite(LED_PIN, level);
10    delay(25);
11  }
12  for (int level = 255; level >= 0; level -= 5) {
13    analogWrite(LED_PIN, level);
14    delay(25);
15  }
16 }
```

MISSION 3

Read ROB's pin map

MISSION 3

The base controller's connections

SOURCE TRAIL — ANALYZING NOW

File: ROBArduino/ROBOT_CEREBELLULAR_BASE_APP/ROBOT_CEREBELLULAR_BASE_APP.ino

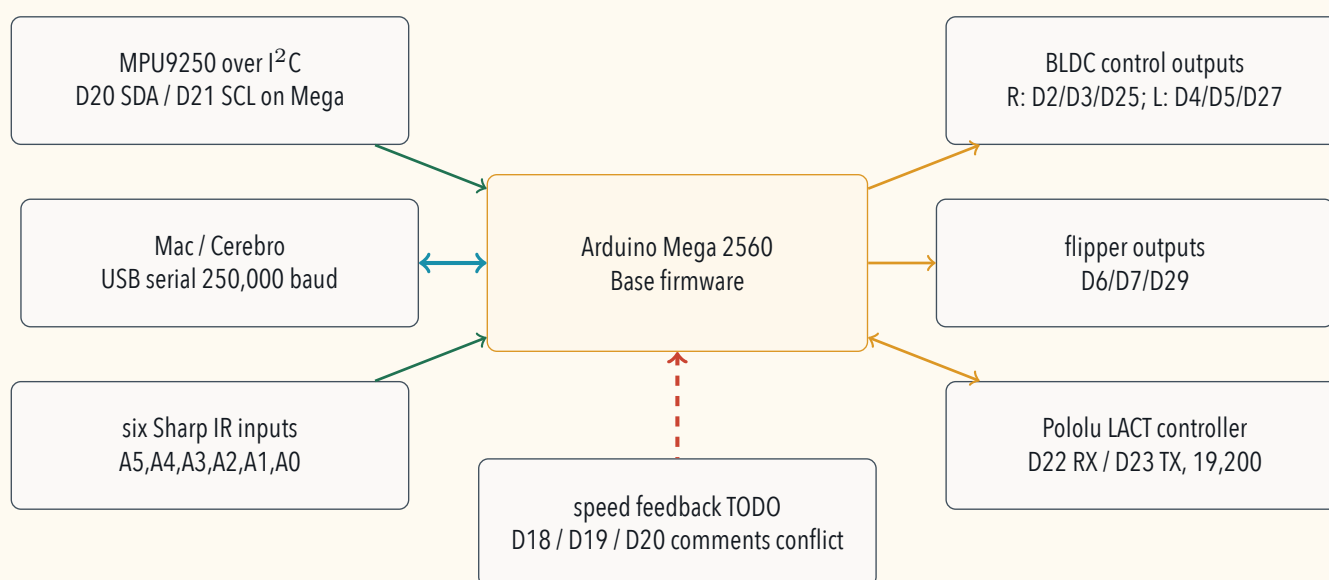
Why it is here: This is the present low-level firmware reference analyzed for pins, serial parsing, treads, flipper, actuator, IR inputs, IMU behavior, and watchdog logic.

Why the archive contains three names

Earlier ROB software divided work among sketches named **Base**, **Torso**, and **Head**. Base handled tread, flipper, actuator, IR, and IMU work. The retired Torso sketch sent fourteen servo targets toward a Maestro for head and arm channels. The retired Head sketch experimented with IMU and PIR sensing. Present-day ROB uses only the Base code, according to the builder. The other two files teach us how the design evolved; they are not instructions to find or connect two more boards.

The two Torso files in ROBArduino have the same SHA-256 hash, so they are byte-for-byte duplicates rather than different firmware versions. A filename can look like a revision even when its contents are identical—another reason engineers use hashes and release records.

The checked-in firmware names these connections. Pin numbers are facts about the file, not a verified public wiring harness. The builder identifies the board as an Arduino Mega 2560. Arduino's current documentation specifies **54 digital I/O pins**, not 55, with 15 PWM outputs, 16 analog inputs, and four hardware serial ports. The board revision, core, installed libraries, and flashed firmware hash still need to be recorded.



As-written firmware map. Dashed feedback is not implemented. A pin number is not a connector drawing; verify the harness, logic levels, grounds, polarity, and safe boot state at the robot.

SUBSYSTEM	SPEED	BRAKE	DIRECTION	NOTE
Right tread	D2	D3	D25	PWM speed output plus two digital controls
Left tread	D4	D5	D27	PWM speed output plus two digital controls
Flipper	D6	D7	D29	PWM speed output plus two digital controls

SIGNAL	PIN OR RATE	FIRMWARE ROLE
Linear actuator serial RX/TX	D22 / D23	D22 receive is declared but unused; D23 transmits to a simple motor controller at 19,200 baud
Front-left / front-right IR	A5 / A4	Long-range Sharp IR model constant
Left / right IR	A3 / A2	Medium-range Sharp IR model constant
Back-left / back-right IR	A1 / A0	Long-range Sharp IR model constant
Mac-to-Arduino serial	250,000 baud	Commands, logs, sensor text, and the role-specific startup line share the port
IMU	I2C via Wire	MPU9250 accelerometer, gyro, and magnetometer

The existing Base sketch says `BEGIN BASE STARTUP SEQUENCE`; the retired sketches say Head or Torso instead. Cerebro resets each USB serial candidate and listens for the Base line without sending command bytes. This is safer than guessing a changing `/dev/cu.*` name and requires no show-day firmware flash. The label identifies a firmware role; it does not certify the wiring or make a motion test safe.

Without changing that firmware, Cerebro can also recognize its existing front and back IR warning sentences. SceneKit shows a recent warning in red, then changes to “clearance unknown” rather than claiming the path is clear. Grass and uneven terrain can fool IR sensors; RPLidar room geometry, direct observation, and the physical stop remain separate layers.

FROM ROB'S BUILD LOG

The source contains TODOs for three motor speed-sensor inputs, ramped motion, and voltage measurement. The comments disagree about left/right encoder assignment, and proposed D20 conflicts with the likely Mega I2C SDA pin used by the active IMU. A TODO is evidence of intent, not evidence that the feature works.

A reviewed interrupt-first Mega proposal

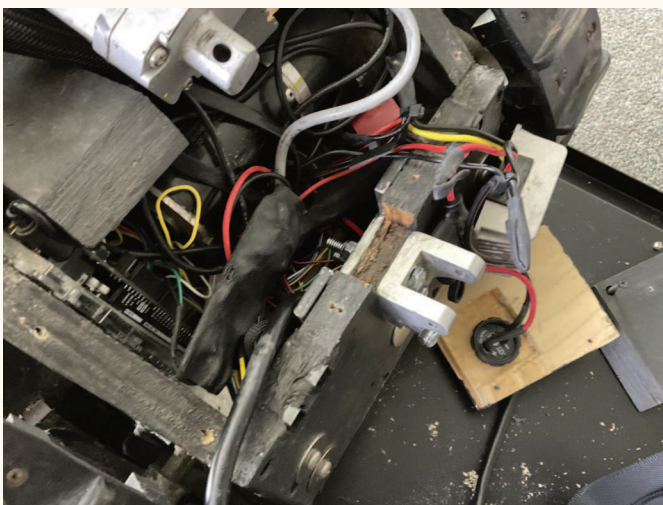
The original wiring spent D2 and D3 on right-motor outputs even though the Mega's six external-interrupt pins are D2, D3, D18, D19, D20, and D21. Long I²C wiring also made the MPU9250 unreliable in ROB's electrically noisy body. The table below is a **proposed harness revision**, not the installed wiring:

ROLE	PROPOSED PIN	REASON
Right / left / flipper tach	D18 / D19 / D2	Three dedicated interrupt inputs; use <code>digitalPinToInterrupt(pin)</code> and keep each ISR tiny.
Right speed / brake / direction	D8 / D24 / D25	Moves the right outputs away from D2/D3; D8 retains PWM.
Left speed / brake / direction	D4 / D5 / D27	Keeps the current left outputs.
Flipper speed / brake / direction	D6 / D7 / D29	Keeps the current flipper outputs.
LACT serial TX / RX	D16 / D17	Uses hardware <code>Serial2</code> instead of timing-sensitive <code>SoftwareSerial</code> .
Short local I²C only	D20 / D21	Reserved for SDA/SCL; do not share D20 with a tach.
IR array	A5 through A0	Preserve all six inputs, but publish filtered telemetry instead of giving one noisy sample motion authority.

Each interrupt should only capture an edge count and timestamp into `volatile` state. The ordinary loop can atomically copy those counters, compute speed over a known interval, reject implausible jumps, publish age and quality, and compare commanded with measured motion. Never print, allocate memory, or run motor policy inside the ISR.

SUCCESSOR CONTROLLER CANDIDATE

The Arduino GIGA R1 WiFi is a credible research successor because its official specification lists 76 GPIO, 13 PWM pins, four UARTs, dual Cortex-M7/M4 cores, and a CAN controller that requires an external transceiver. It uses 3.3 V logic and is therefore *not* a drop-in Mega replacement. Its base board has no integrated IMU; the separate GIGA Display Shield adds a BMI270 over I²C, which does not solve a long noisy harness by itself. Begin a migration with D2/D3/D18 as tach candidates, D4/D5/D6 as PWM candidates, D16/D17 for the LACT UART, and D93/D94 only behind an approved CAN transceiver. Compile an `attachInterrupt` smoke test, measure every logic level and startup state, and prove latency under load before moving one live actuator.



An overhead chassis view shows why a pin table alone is not enough: routing, connectors, power separation, and service access matter.



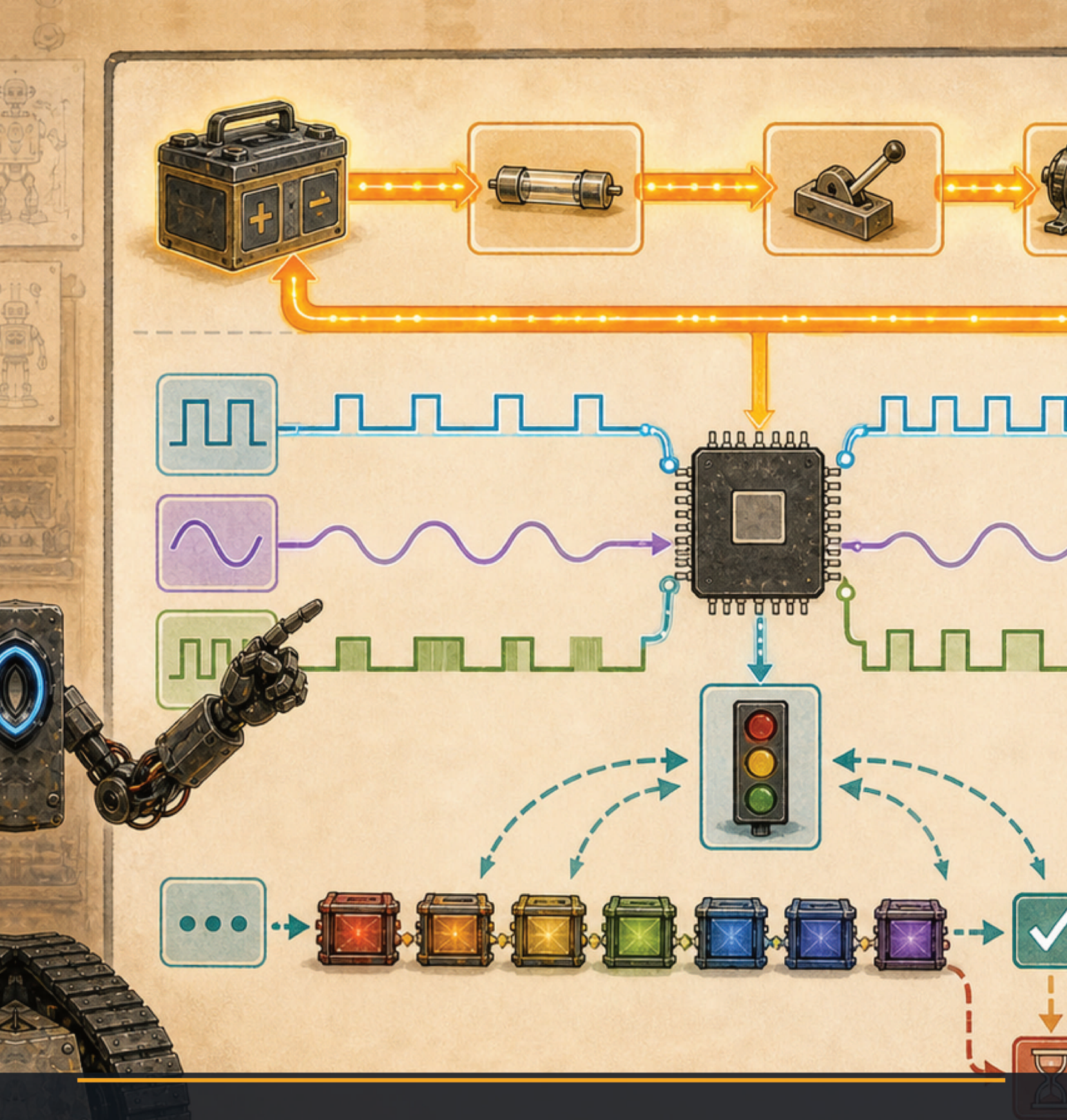
Blue tape labels show an important maintenance habit. A durable harness plan should replace temporary labels before public operation.

FIRMWARE PINS ARE NOT A HARNESS DRAWING

The source preserves pin names used by a historical Arduino Mega 2560 Base sketch. The present connectors, conductor colors and gauges, logic levels, tach edges, strain relief, and branch labels have not been traced for publication, and no successor board map has been adopted in this edition. Treat the table as a code-reading exercise only. Do not connect hardware from it.



A close view of an earlier wiring revision shows why every connector needs a name, destination, expected signal, safe state, and service plan.



MISSION 4

A command is a tiny data structure

MISSION 4

Decode the seven-field message

SOURCE TRAIL — ANALYZING NOW

File: Cerebro/Cerebro/ROBSerialBox.m

Why it is here: Read this beside the Base sketch: the Mac writes the text frame and the Arduino parses it. Neither half alone defines the complete exchange.

The base firmware waits for a tilde character, then reads seven signed, fixed-width text fields separated by commas:

```
~+0000,+0100,+0000,+0100,+0000,+0000,+0000
```

The positions are:

1. left tread brake;
2. left tread signed speed;
3. right tread brake;
4. right tread signed speed;
5. flipper brake;
6. flipper signed speed;
7. linear actuator signed speed.

The example above releases the brakes and asks both treads for +100, while the flipper and actuator remain at zero. The entire frame is 42 characters when every field is exactly five characters.

Differential steering

Two treads can steer without a steering wheel. If both sides move forward at the same speed, the robot travels roughly straight. If one moves forward while the other moves backward, the robot rotates.

INTENT	LEFT	RIGHT	IDEA
Forward	+100	+100	Same direction and magnitude
Backward	-100	-100	Same reverse direction
Turn right	+100	-100	Counter-rotating treads
Turn left	-100	+100	Opposite counter-rotation
Gentle curve	+70	+35	Outside tread travels faster
Stop	0	0	Zero requested speed

COMPUTER SCIENCE CONNECTION

The message is a **protocol**: an agreement about order, spelling, valid ranges, and meaning. Protocols let programs written in different languages cooperate. The Mac code can be Objective-C or Swift while the Arduino code is C++, yet both can understand the same frame.

Build a safer parser on paper

The historical parser teaches useful ideas, but it is not a model for finished safety-critical code. It reads when only one byte is known to be available, does not prove all 42 characters arrived, does not validate comma positions or numeric ranges, and stores five characters without a sixth null terminator before calling `atoi`. A damaged frame could be misread.

A safer receiving state machine would:

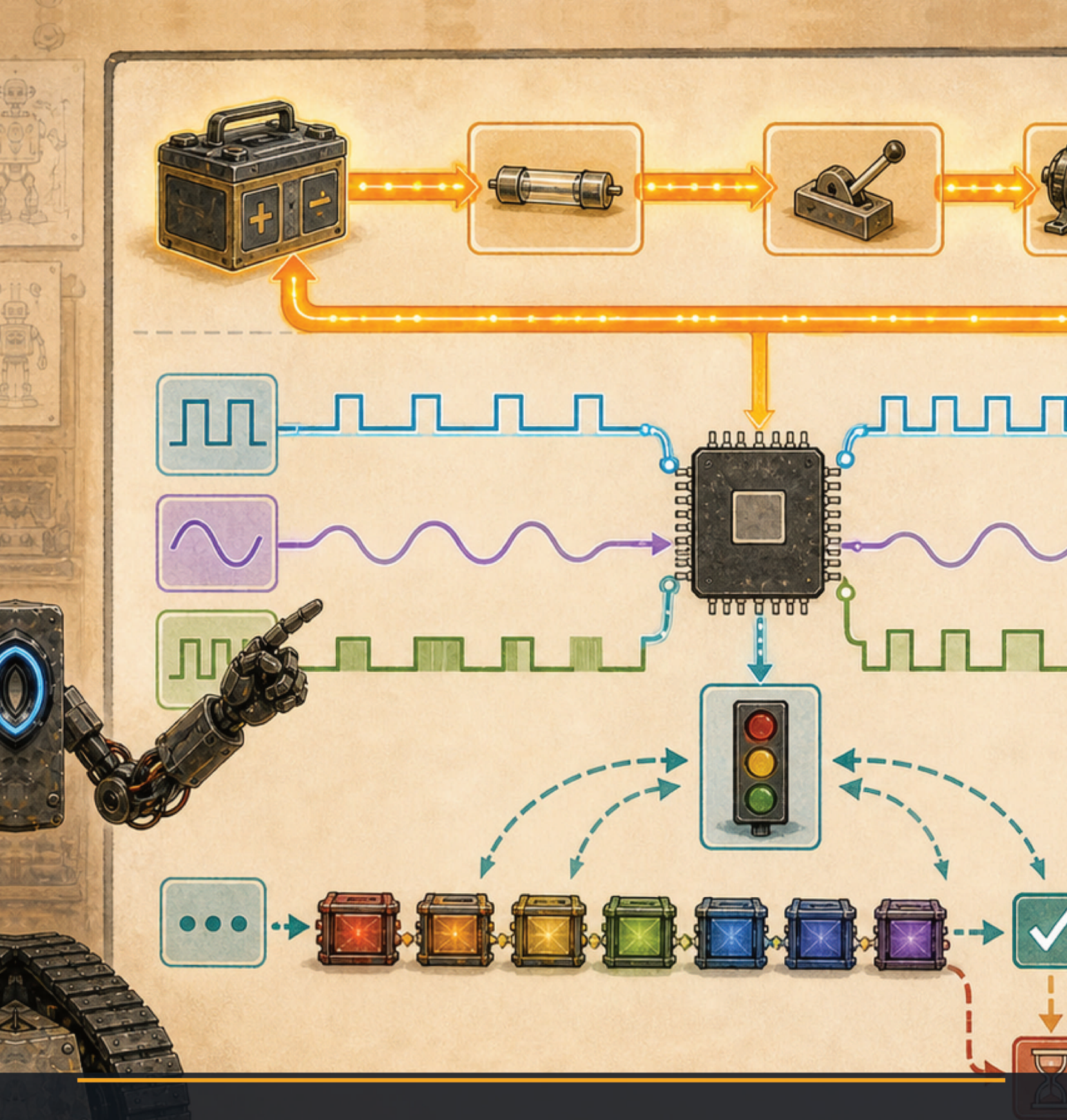
1. search for the start marker;
2. collect exactly one bounded frame;
3. verify all separators and signs;
4. convert each field only after validation;
5. reject values outside the allowed range;
6. update all outputs atomically, or update none;
7. command a safe stop after a deadline.

Listing 4.1: Pseudocode for an all-or-nothing command decoder.

```
1 if (completeFrameAvailable()) {  
2   Frame candidate = readBoundedFrame();  
3   if (candidate.isValidShape() &&  
4     candidate.valuesAreInRange()) {  
5     lastGoodCommand = candidate;  
6     lastGoodTime = millis();  
7   } else {  
8     reportBadFrame();  
9   }  
10 }  
11  
12 if (millis() - lastGoodTime > COMMAND_DEADLINE_MS) {  
13   commandNeutralAndBrake();  
14 }
```

FRAME INSPECTOR

Write five valid and five invalid frames. Include a missing comma, a wrong sign, an extra digit, a letter, and an out-of-range speed. Trade with a partner and circle the first rule each invalid frame breaks.



MISSION 5

Sensors are measurements, not truth

MISSION 5

IR distance and the IMU

The source declares six IR objects, but the active warning routine reads only the front and rear pairs. It compares those four readings with 25 cm and counts repeated low readings before announcing a shutdown warning. The left/right pair appears only in commented telemetry.

That is a simple **temporal filter**: require evidence across several samples. It can reduce single-sample glitches, but it does not solve every problem. Grass, angled surfaces, sunlight, dark material, shiny material, sensor placement, and reflections from the floor can change optical range data.

FROM ROB'S BUILD LOG

The call that reads the IR sensors is active in the current loop. The assignments that would set the forward and backward shutdown flags are commented out. The code therefore still reads and reports warnings, but does not currently assert those movement-inhibit flags. The builder reports disabling the behavior after many grass-triggered stops. The final policy and calibration need measurement before publication as a safety feature.

Three sensors become orientation

The MPU9250 combines three sensor families:

- accelerometer: linear acceleration and gravity;
- gyroscope: rotation rate;
- magnetometer: magnetic field direction.

The firmware uses a Mahony quaternion filter, then prints yaw, pitch, and roll. This is **sensor fusion**: combine imperfect measurements to estimate a state no single sensor supplies reliably.

FILTER A DATA SET

Record ten distance readings from a safe classroom sensor aimed at a box. Compute the mean and median. Move the box, record ten more, and graph both groups. Which summary handles one wild reading better? Do not turn the result into an automatic stop rule without testing more surfaces and conditions.



MISSION 6

Silence must become stop

MISSION 6

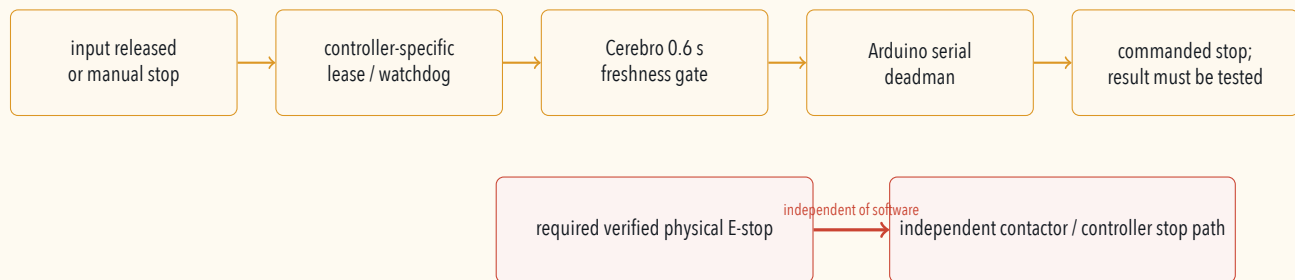
Watchdogs and fail-safe states

ROB's Arduino loop counts down a two-stage heartbeat. When it expires, the firmware calls the same motor-command function with zero tread, zero flipper, and zero actuator values. However, the timeout passes brake fields of zero, which the code interprets as released brakes. The physical result may be coast rather than hold and must be measured.

The implementation counts loop iterations before checking the second counter, so the exact deadline changes when other loop work changes. It also refreshes the deadline for any received byte before validating a complete frame; malformed traffic could therefore keep an earlier command alive. A production version should refresh only after a valid frame, measure elapsed monotonic time directly, define the safe brake state precisely, and test failures under load.

COMPUTER SCIENCE CONNECTION

A **watchdog** asks, "Did the expected event happen before the deadline?" A **fail-safe state** is the state chosen when the answer is no. For a small lamp, OFF may be safe. For a heavy robot on a slope, braking, de-energizing, and mechanical holding behavior must be engineered and tested.



SAFETY CHECK

The yellow emergency-stop hardware visible in the 2025 bench photograph is not enough to prove a complete emergency-stop circuit. A public robot needs an independently verified stop architecture, clear operator reach, reset rules, contactor or controller behavior appropriate to the hazards, and documented tests.

WATCHDOG WITHOUT MOTORS

Program an Arduino LED to remain lit only while a serial "K" arrives at least once each second. Stop the sender and confirm that the LED goes dark. Add a latched error LED that requires a deliberate reset. This demonstrates the timing idea without motion.

Listing 6.1: A classroom heartbeat pattern using an LED.

```
1 const unsigned long DEADLINE_MS = 1000;
2 unsigned long lastHeartbeat = 0;
3 bool heartbeatSeen = false;
4
5 void setup() {
6   pinMode(LED_BUILTIN, OUTPUT);
```

```
7  digitalWrite(LED_BUILTIN, LOW); // boot stale
8  Serial.begin(9600);             // classroom sender
9  }
10
11 void loop() {
12   if (Serial.available() && Serial.read() == 'K') {
13     lastHeartbeat = millis();
14     heartbeatSeen = true;
15   }
16   bool fresh = heartbeatSeen &&
17             millis() - lastHeartbeat <= DEADLINE_MS;
18   digitalWrite(LED_BUILTIN, fresh ? HIGH : LOW);
19 }
```



Software deadlines and a physical emergency stop solve different failure problems. The visible button is evidence of hardware, not proof that the complete stop circuit has been verified.

MISSION 7

Read old code like an engineer

SOURCE TRAIL — ANALYZING NOW

File: ROBArduino/ROBOT_CEREBELLULAR_TORSO_APP/ROBOT_CEREBELLULAR_TORSO_APP.ino

Why it is here: This retired sketch is historical open-source evidence from the former three-Arduino architecture; it is not instructions to install another active controller.

Historical code is a laboratory notebook. It preserves successful ideas, unfinished experiments, and mistakes. Respect it by asking precise questions.

OBSERVATION	ENGINEERING QUESTION
The rotation test repeats the same Boolean condition twice.	Was the second branch intended to describe the opposite turn? Add unit tests before changing behavior.
Some method names, direction comments, and printed labels disagree.	Which direction does each physical tread actually move with each pin state? Record a wheels-off-ground test.
Values are not clamped before PWM.	What are the allowed command ranges, and what happens to malformed values?
The IR inhibit assignments are commented.	What operating conditions caused false triggers, and what independent protections remain?
Ramping and speed feedback are TODOs.	What acceleration, deceleration, encoder, and stopping-distance requirements should a new version meet?
The actuator accepts special values 3201 and -3201 to clear safe start.	Should reset be a separate typed command instead of overloading speed?

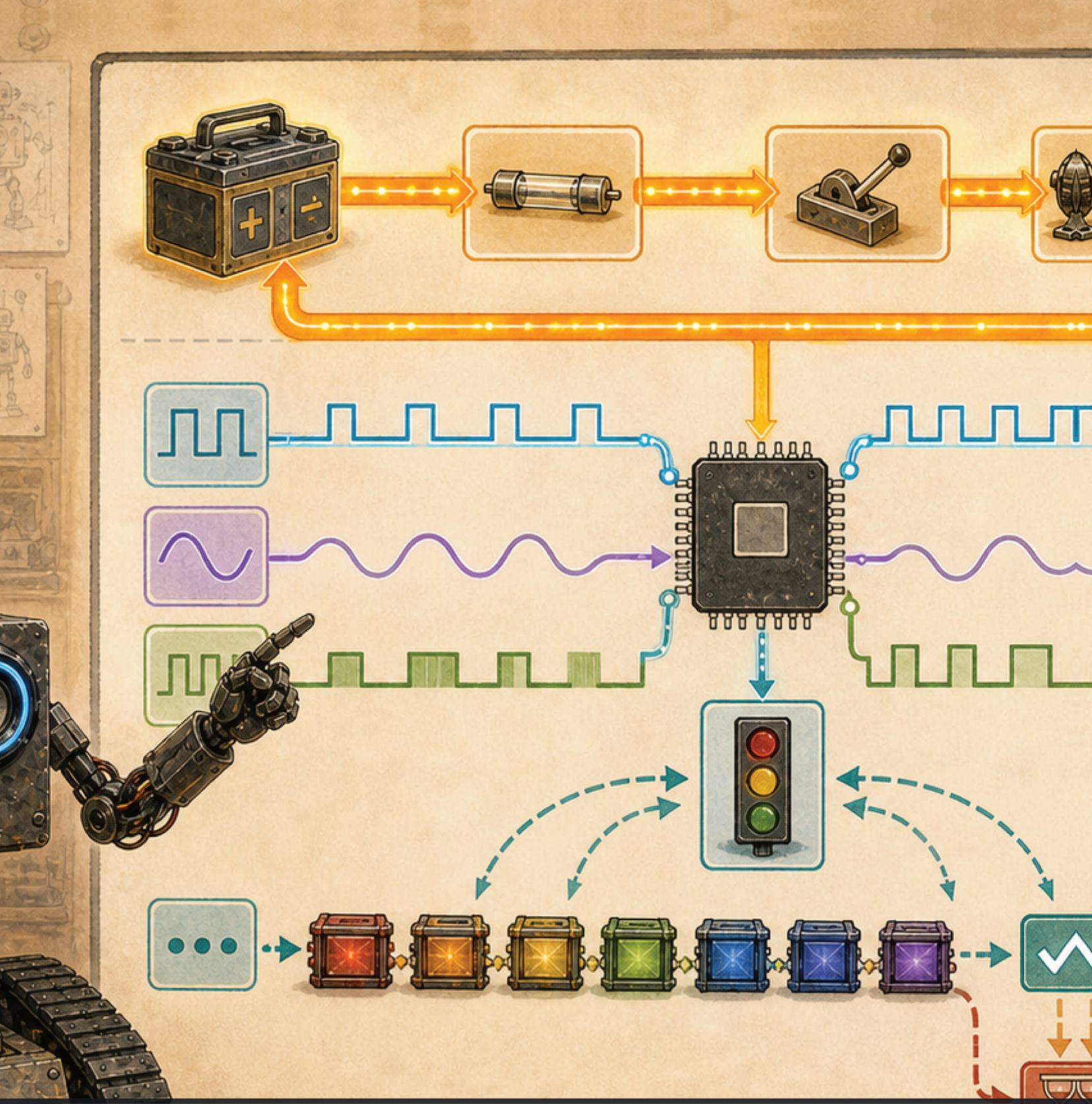
ADVANCED CHANNEL

Before modernizing the firmware, build a byte-for-byte protocol fixture and a hardware-in-the-loop test stand with the drive load isolated. Characterize the existing behavior first. A refactor that looks cleaner can still reverse a tread or release a brake.

WHAT THE RECORD CAN AND CANNOT ESTABLISH

The builder identifies the controller platform as an Arduino Mega 2560, and the repository preserves the Base sketch. It does not establish the flashed hash, installed libraries, physical motor polarity, controller safe-start configuration, measured heartbeat deadline, or calibrated IR performance of the

present robot. Grass and outdoor operation exposed traction and sensing limits, but no dated test report survives. Readers may analyze the defensive software pattern; only a restrained local test can establish physical behavior.



DEEP LAB

Energy moves; signals describe

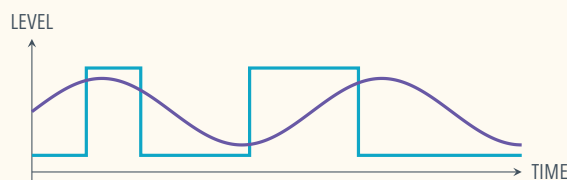
MISSION 8

Build a mental oscilloscope

Voltage is a difference in electric potential between two points. Current is charge flow through a path. Resistance relates voltage and current for many components, but real motors, batteries, semiconductors, and sensors also change with time, temperature, and load. A signal uses a changing electrical quantity to represent information; the receiving circuit must agree on reference, range, timing, and meaning.

Read a graph as a story in time

An oscilloscope draws voltage vertically and time horizontally. A digital signal spends most of its time near two allowed ranges. An analog signal can take many values. PWM is digital switching whose duty cycle—the fraction of each period spent active—changes an average effect. Frequency answers how often a pattern repeats; duty cycle answers how much of each repetition is active.



WAVEFORM DETECTIVE

Draw three PWM traces with the same period and duty cycles near one quarter, one half, and three quarters. Without using a motor, represent their average effect by shading the active area. Then draw two traces with equal duty cycle but different frequency. Explain what stayed the same and what changed.

A circuit needs a complete question

“Is this pin high?” is incomplete until we know high relative to what reference, at which voltage, measured by which device, and at what time. “Can this output drive a motor?” is usually the wrong connection: a logic pin communicates intent to a properly designed driver; it does not supply traction power.

SAFETY CHECK

Use only an educator-approved, current-limited low-voltage learning kit. Disconnect power before changing wiring. Never connect a classroom board to ROB, a wall outlet, an unknown battery, a motor controller, or a high-current load.

MISSION 9

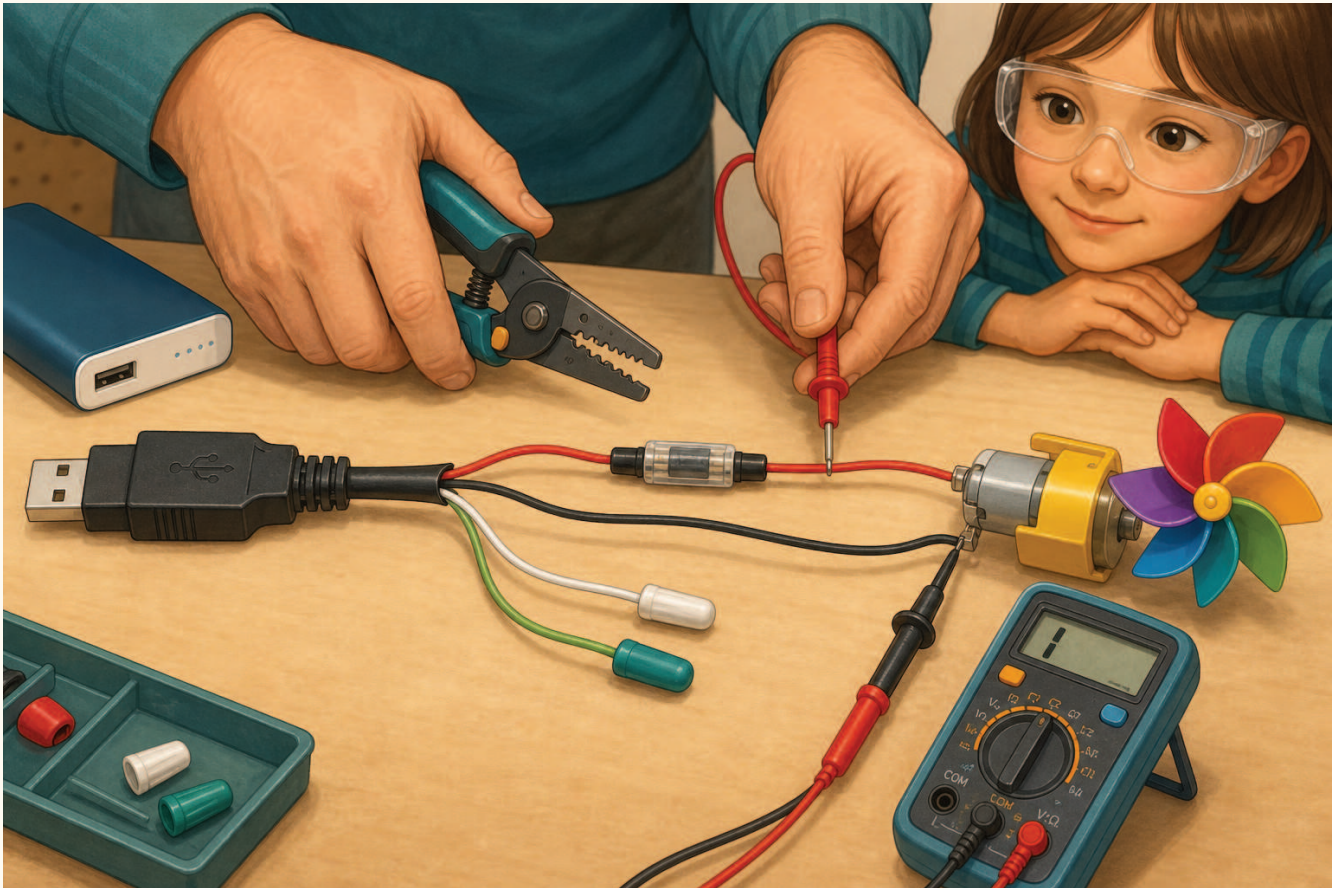
Give a toy a new 5 V power path

A USB cable can carry both electrical energy and information. In a common USB 2.0 cable, one conductor supplies about 5 V, one is the return path called GND, and two carry data. At the Maker Faire, a prepared cable lets us expose those roles and use the power pair to run a small toy that would normally use batteries.

This is an experiment in **energy conversion**: the USB source provides electrical energy, the toy changes it into motion, light, or sound, and the ground conductor completes the circuit. It is also an experiment in evidence. Common color conventions are clues, not proof. A cable may use unexpected colors, contain extra wires, or omit data wires entirely, so a mentor must identify and measure the conductors before a learner connects a load.

UNPLUG, PREPARE, VERIFY, THEN ENERGIZE

Only an adult mentor cuts and strips a sacrificial cable, and the cable remains unplugged during all cutting, stripping, separating, and insulating. Never cut a cable connected to a power bank, computer, charger, or wall outlet. Use only the event's approved 5 V current-limited USB source and a known low-current toy rated for approximately 5 V. Never use ROB's wiring, an unknown cable, USB-C, a damaged power source, lithium cells, or a toy with batteries still installed. Stop for heat, odor, noise, damaged insulation, or unexpected behavior.



A mentor prepares and verifies the cable while it is unplugged. Red and black show the usual 5 V and GND convention; the meter, not the color, establishes the result. The unused data conductors are insulated separately.

What is hiding inside the cable?

The familiar colors in many USB 2.0 cables are:

USUAL COLOR	USUAL ROLE	WHAT WE DO IN THIS LAB
Red	VBUS, about +5 V	Mentor verifies it, adds current protection, and connects it to the toy's positive input.
Black	GND, return	Mentor verifies it and connects it to the toy's negative input.
White	Data —	Do not connect to the toy; insulate its end by itself.
Green	Data +	Do not connect to the toy; insulate its end by itself.
Bare foil, braid, or drain	Shield	Trim or secure it as the mentor's cable plan specifies; never let it touch a conductor.

WHY TWO WIRES ARE ENOUGH FOR THIS TOY

The toy needs a potential difference across its two power terminals. The 5 V conductor is the outward energy path and GND is the return path. The data pair is unnecessary for a simple power-only load. This does *not* mean every USB device works from two wires: many devices communicate, negotiate power, or require their original electronics.

Match the source to the load

Read the toy's label or battery compartment before connecting anything. Three alkaline cells are commonly described as 4.5 V, which may make some simple toys candidates for a carefully reviewed 5 V demonstration. A two-cell 3 V toy is **not** automatically a 5 V toy. The mentor must approve the voltage range and estimate or measure the running and startup current. Small motors can briefly draw much more current when starting or stalled.

Power is the rate of energy transfer:

$$P = VI.$$

If a toy uses 0.20 A from a 5.0 V source, its input power is approximately

$$P = (5.0\text{ V})(0.20\text{ A}) = 1.0\text{ W}.$$

The source's current rating is a maximum capability, not current that it forces through every load. The load and circuit determine the current. Protection must still interrupt a short circuit, and the event source must tolerate the motor's startup current without shutting down or overheating.

Maker Faire mentor-and-learner procedure

1. **Choose and label.** The mentor selects a sacrificial USB-A cable, a switch or clip lead set with covered contacts, an inline fuse or other approved current protection, and a toy whose voltage and current requirements are known. Mark the cable "5 V LAB ONLY."
2. **Predict while unplugged.** The learner sketches source → protection → switch → toy → return and predicts which energy conversion will be visible.
3. **Open while unplugged.** At an adult-only tool station, the mentor removes a short section of outer jacket without nicking conductor insulation, separates the conductors, and exposes only the tiny length needed at the two power leads. A USB breakout adapter is the preferred reusable alternative when available.
4. **Isolate unused wires.** The mentor caps the data conductors individually, secures any shield, adds strain relief, and ensures no bare pieces can touch.
5. **Verify before the toy.** With the prepared ends separated and enclosed against accidental contact, the mentor connects the cable to the approved current-limited source and measures voltage and polarity with a meter. Disconnect again before attaching the toy.
6. **Remove batteries.** Confirm the toy's batteries are absent. Never connect USB power in parallel with installed cells or the toy's charging circuit.
7. **Connect without power.** Match verified +5 V to the toy's positive terminal and verified GND to its negative terminal. Cover every connection and provide strain relief. A mentor performs a continuity and short-circuit check.
8. **Run a bounded trial.** Place the toy so motion cannot catch hair, clothing, or fingers. Energize for a few seconds, observe, then switch off and disconnect. The learner does not hold the toy or exposed wiring while it is powered.
9. **Explain the evidence.** Record measured voltage, observed motion/light/sound, and any change from the prediction. Explain why opening either conductor stops the complete path.

THE LEARNER'S JOB

Point to the source, protection, switch, load, and return path. Before power is connected, predict what will happen if the circuit is complete, the switch is open, or the polarity is reversed. The reverse-polarity case is a paper prediction only; do not try it on the toy. After the safe trial, draw arrows for conventional current and write one sentence about where the energy went.

MEASURE, DO NOT GUESS

Create a four-column lab record: **question**, **prediction**, **measurement or observation**, and **explanation**. Include the source voltage before connection, voltage while the toy runs, current if the mentor's approved setup measures it safely, and whether the source or toy became warm. A useful result can be "the source shut down"—that is evidence of a limit, not permission to bypass protection.

MISSION 10

Messages need grammar and time

A serial frame is a sentence with a grammar. The receiver needs a start rule, fields, separators or lengths, valid character rules, numeric bounds, and an end rule. A checksum can detect many accidental changes but does not prove identity or permission. Authentication, authorization, freshness, and physical bounds answer different questions.

QUESTION	CHECK
Where does it begin?	Magic bytes, delimiter, or length-prefixed header.
Is it intact?	Exact length, syntax, and optional error-detecting code.
Is each value meaningful?	Type, finite-number, range, and relationship checks.
Is it new?	Sequence number, timestamp, and expiring lease.
Who sent it?	Cryptographic authentication bound to a device identity.
May that device do this?	Server-owned role and per-message authorization.
What if it stops?	Watchdog and defined safe state.

HUMAN PARSER LAB

Invent a paper message for a two-light robot: start symbol, sequence number, red level, blue level, and end symbol. Write five valid cards and five adversarial cards: missing field, duplicate sequence, extra character, out-of-range value, and stale timestamp. Exchange decks. The receiver must reject a whole invalid card without applying half of it.

STATE MACHINES BEAT WISHFUL PARSING

A parser can be modeled as states such as `WaitingForStart`, `ReadingHeader`, `ReadingPayload`, `Validating`, `Accepted`, and `Rejected`. Every byte causes one defined transition. Timeouts and oversize lengths lead to `Rejected`. Only `Accepted` may update command state. Draw the transitions before writing code; missing arrows expose ambiguous behavior.

Field words

BAUD Symbol rate used by a serial link.

CURRENT Rate of electric charge flow.

FRAME One bounded protocol message.

GROUND A chosen circuit reference node.

IMU A motion-sensing unit containing inertial sensors.

NULL TERMINATOR The zero byte that ends a C

string.

PWM Timed pulses whose duty cycle controls an average effect.

RANGE CHECK A test that rejects values outside allowed limits.

SERIAL Sending symbols in sequence over a link.

WATCHDOG A deadline monitor for an expected update.

FINAL CHECK

Can you separate a power path from a signal path? Can you decode the seven fields? Can you explain why a complete frame must be validated before any motor changes? Can you describe why IR readings are evidence rather than truth? Then you are ready for Motion Workshop.

Source trail: ROBArduino Base sketch, especially pin definitions, serial parser, IR checks, keepalive, and motor mapping; retired Torso and Head sketches for architecture history; ROB_v3.pdf; gallery originals. Arduino archive inspected 2026-08-10. Restricted vendor-document specifications were neither reproduced nor paraphrased.